ITC-C508 Lab Exercise WW1 and WW2 (Optimization and GPU Testing)

Submitted by Group: Cache

Members:

- Sasil, Perlin Precious (Group Leader)
- Laylo, Hannah Alexa
- Tuguinay, Nanelle

Submitted to: Doc. Rodolfo Raga

Introduction

- The aim of this experiment was to identify the most effective RAG configuration for the Barangay VAW Desk Handbook and Republic Act No. 9710 (otherwise known as the Magna Carta of Women), as well as to observe the differences between CPU and GPU execution time. Both the Barangay VAW Desk Handbook and Republic Act No. 9710 were the sources used for training the chatbot on how to help victims of domestic violence, give information about women's rights, and about some provisions of law. Four different configurations were created using different values of chunk size and chunk overlap but keeping the same Top-K value.
- For each configuration, the documents' sources, the test questions, the embedding model, the language model, and the generation settings remained unchanged, which helped establish a baseline for the comparison. The experiment compared the results obtained with the help of the retrieval metrics such as Precision@3, Recall@3, and Mean Reciprocal Rank at 3 (MRR@3). Besides that, the answers produced by the model were analyzed in terms of their precision, relevancy to the retrieved passages, and compliance with the expected answers. Finally, indexing time, average response time, and total time needed to run the experiments were taken into consideration to evaluate the difference in CPU and GPU performance.

DATASET: Barangay VAW Desk Handbook (2023)

- The first dataset used in this experiment is the *Barangay VAW Desk Handbook (2023)*. The handbook provides information and guidelines for Barangay Violence Against Women (VAW) Desk officers in responding to cases of violence against women and their children. It covers the roles and responsibilities of VAW Desk officers, initial assistance and assessment, proper documentation, interview procedures, referral mechanisms, and information about the rights and legal remedies available to victim-survivors, including protection orders.
- For this experiment, the handbook was processed and converted into text that could be divided into smaller chunks for embedding and retrieval. These chunks were indexed in a vector database to allow the chatbot to retrieve relevant passages based on users' questions. The retrieved passages serve as contextual references for generating answers, helping ensure that responses are grounded in the information provided by the handbook rather than relying solely on the language model's pretrained knowledge.
- This dataset was used to evaluate how different chunk sizes and overlap values affect retrieval relevance, answer correctness, response generation time, and overall chatbot performance. The experiment also compared CPU and GPU execution to examine the effects of hardware acceleration on processing time.

DATASET: Republic Act No. 9710 – Magna Carta of Women

- The second dataset used in this experiment is *Republic Act No. 9710*, also known as the *Magna Carta of Women*. This Philippine law recognizes, protects, and promotes the rights of women, particularly in addressing discrimination and violence against women. It outlines the responsibilities of the government in ensuring women's rights and provides legal provisions concerning protection from violence, access to services, and mechanisms for addressing women's concerns.
- For this experiment, the document was processed and converted into text to prepare it for chunking, embedding, and retrieval. The resulting chunks were

indexed in a vector database, allowing the chatbot to identify relevant legal provisions based on users' questions. These retrieved passages serve as references for generating responses related to the rights of women, government responsibilities, and other provisions established under the law.

- This dataset was used to assess the chatbot's ability to retrieve and provide relevant information from a legal document. Different chunk size and overlap configurations were evaluated to determine their effects on retrieval performance and the quality of generated answers. The same evaluation process and CPU–GPU performance comparison were applied to maintain consistency across both datasets.

Executable RAG Implementation

- The following is the full implementation of the Retrieval Augmented Generation (RAG) system for the chatbot using the Barangay VAW Desk Handbook (2023) and Republic Act No. 9710 (Magna Carta of Women) as the core knowledge base. The process involves document preparation, text splitting into chunks, embeddings generation, vector database creation, information retrieval, and answer generation. The implementation enables one to test various chunk sizes and overlaps and benchmark CPU and GPU performances.

```
# ============================================================
# RAG DOCUMENT PROCESSING PIPELINE
# PDF EXTRACTION, CLEANING, CHUNKING, EXCEL EXPORT,
# EMBEDDINGS, FAISS, AND RETRIEVAL
# Google Colab implementation
# ============================================================


# ============================================================
# 1. INSTALL REQUIRED LIBRARIES
# ============================================================

!pip install -q faiss-cpu sentence-transformers pypdf transformers


# ============================================================
# 2. IMPORT LIBRARIES
# ============================================================
```

```
import time
from google.colab import files
from pypdf import PdfReader
from sentence_transformers import SentenceTransformer

import faiss
import numpy as np
import pandas as pd
import pickle
import re
import os

from openpyxl.styles import Font, PatternFill, Alignment


# ============================================================
# 3. INDEXING PARAMETERS
# ============================================================

CHUNK_SIZE = 800
CHUNK_OVERLAP = 80
TOP_K = 3

EMBEDDING_MODEL_NAME = "all-MiniLM-L6-v2"

# Ignore pages with almost no usable extracted text.
# This helps prevent page numbers and isolated symbols
# from becoming standalone chunks.
MIN_PAGE_TEXT_LENGTH = 20

# Avoid creating tiny final chunks when possible.
MIN_TAIL_LENGTH = 50


# ============================================================
# 4. VALIDATE PARAMETERS
# ============================================================

if CHUNK_SIZE <= 0:
    raise ValueError("CHUNK_SIZE must be greater than 0.")

if CHUNK_OVERLAP < 0 or CHUNK_OVERLAP >= CHUNK_SIZE:
    raise ValueError(
        "CHUNK_OVERLAP must be >= 0 and smaller than CHUNK_SIZE."
    )

if TOP_K <= 0:
    raise ValueError("TOP_K must be greater than 0.")

print("=" * 70)
print("RAG INDEXING CONFIGURATION")
print("=" * 70)
print("Chunk size             :", CHUNK_SIZE)
print("Chunk overlap          :", CHUNK_OVERLAP)
```

```
print("Top-K                  :", TOP_K)
print("Embedding model        :", EMBEDDING_MODEL_NAME)
print("Minimum page text      :", MIN_PAGE_TEXT_LENGTH)
print("Minimum tail length    :", MIN_TAIL_LENGTH)


# ============================================================
# 5. TEXT CLEANING FUNCTIONS
# ============================================================

def clean_pdf_text(text):
    """
    Remove illegal XML control characters that may appear
    in PDF-extracted text.

    Preserve normal whitespace and readable Unicode text.
    """

    if not isinstance(text, str):
        return ""

    # Remove control characters that Excel/XML cannot store.
    text = re.sub(r"[\x00-\x08\x0B\x0C\x0E-\x1F]", "", text)

    # Normalize whitespace into single spaces.
    text = re.sub(r"\s+", " ", text).strip()

    return text


def clean_excel_text(text):
    """
    Extra safety check before writing text to Excel.
    """

    if not isinstance(text, str):
        return text

    return re.sub(r"[\x00-\x08\x0B\x0C\x0E-\x1F]", "", text)


# ============================================================
# 6. UPLOAD PDF DOCUMENT
# ============================================================

print("\n" + "=" * 70)
print("UPLOAD DOCUMENT")
print("=" * 70)

uploaded = files.upload()

if not uploaded:
    raise RuntimeError("No file was uploaded.")
```

```
pdf_file = next(iter(uploaded.keys()))

if not pdf_file.lower().endswith(".pdf"):
    raise ValueError("Please upload a PDF file.")

print("\nSelected file:", pdf_file)

script_start_time = time.time()

# ============================================================
# 7. EXTRACT AND CLEAN PDF TEXT
# ============================================================

print("\n" + "=" * 70)
print("EXTRACTING AND CLEANING PDF TEXT")
print("=" * 70)

reader = PdfReader(pdf_file)

pages = []
empty_pages = []
short_pages = []

for page_number, page in enumerate(reader.pages, start=1):

    try:
        raw_text = page.extract_text() or ""
    except Exception as error:
        print(f"Warning: Could not extract page {page_number}: {err
        raw_text = ""

    page_text = clean_pdf_text(raw_text)

    if not page_text:
        empty_pages.append(page_number)
        continue

    if len(page_text) < MIN_PAGE_TEXT_LENGTH:
        short_pages.append({
            "page": page_number,
            "text": page_text
        })
        continue

    pages.append({
        "page": page_number,
        "text": page_text
    })


print("Total PDF pages             :", len(reader.pages))
print("Pages with usable text      :", len(pages))
print("Empty/unextractable pages   :", len(empty_pages))
print("Very short pages skipped    :", len(short_pages))
```

```
if short_pages:
    print("\nShort pages skipped:")
    for item in short_pages[:15]:
        print(f"Page {item['page']}: {repr(item['text'])}")

if not pages:
    raise RuntimeError(
        "No usable text was extracted. "
        "The PDF may require OCR."
    )


# ============================================================
# 8. CREATE CHUNKS
# ============================================================

def create_chunks(
    pages,
    chunk_size,
    chunk_overlap,
    min_tail_length=50
):
    """
    Split each page independently into overlapping chunks.

    Each chunk retains its original PDF page number.

    If the final remainder is very short, append it to the
    preceding chunk instead of creating a tiny fragment.
    """

    chunks = []
    chunk_id = 0

    step_size = chunk_size - chunk_overlap

    for page_data in pages:

        page_number = page_data["page"]
        page_text = page_data["text"]

        start = 0
        page_chunk_positions = []

        while start < len(page_text):

            remaining = len(page_text) - start

            # Avoid an extremely short final chunk.
            # Extend the previous chunk to include the remainder.
            if remaining < min_tail_length and page_chunk_positions
                previous_position = page_chunk_positions[-1]
                previous_chunk = chunks[previous_position]
```

```
                previous_start = previous_chunk["_start"]
                previous_chunk["text"] = page_text[
                    previous_start:
                ].strip()

                break

            end = min(start + chunk_size, len(page_text))
            chunk_text = page_text[start:end].strip()

            if chunk_text:
                chunks.append({
                    "chunk_id": chunk_id,
                    "page": page_number,
                    "text": chunk_text,
                    "_start": start
                })

                page_chunk_positions.append(len(chunks) - 1)
                chunk_id += 1

            start += step_size

    # Remove internal processing metadata.
    for chunk in chunks:
        chunk.pop("_start", None)

    return chunks


chunks = create_chunks(
    pages,
    CHUNK_SIZE,
    CHUNK_OVERLAP,
    MIN_TAIL_LENGTH
)

if not chunks:
    raise RuntimeError("No chunks were generated.")


# ============================================================
# 9. DISPLAY CHUNKING RESULTS
# ============================================================

print("\n" + "=" * 70)
print("CHUNKING RESULTS")
print("=" * 70)

chunk_lengths = [len(chunk["text"]) for chunk in chunks]

print("Number of chunks             :", len(chunks))
print("Average characters per chunk :", round(np.mean(chunk_lengths
```

```
print("Shortest chunk               :", min(chunk_lengths))
print("Longest chunk                :", max(chunk_lengths))

tiny_chunks = [
    chunk for chunk in chunks
    if len(chunk["text"].strip()) < MIN_TAIL_LENGTH
]

print("Chunks shorter than 50 chars:", len(tiny_chunks))

if tiny_chunks:
    print("\nShort chunks for inspection:")
    for chunk in tiny_chunks[:10]:
        print(
            f"Chunk {chunk['chunk_id']} | "
            f"Page {chunk['page']} | "
            f"{repr(chunk['text'])}"
        )


# ============================================================
# 10. EXPORT CHUNKS TO EXCEL
# ============================================================

print("\n" + "=" * 70)
print("EXPORTING CHUNKS TO EXCEL")
print("=" * 70)

chunk_records = []

for chunk in chunks:
    chunk_records.append({
        "Chunk_ID": chunk["chunk_id"],
        "Source_Document": pdf_file,
        "Page_Number": chunk["page"],
        "Character_Count": len(chunk["text"]),
        "Chunk_Size_Setting": CHUNK_SIZE,
        "Chunk_Overlap_Setting": CHUNK_OVERLAP,
        "Chunk_Text": clean_excel_text(chunk["text"])
    })

chunks_df = pd.DataFrame(chunk_records)

base_name = os.path.splitext(os.path.basename(pdf_file))[0]
EXCEL_FILE = f"{base_name}_chunks_inspection.xlsx"

with pd.ExcelWriter(EXCEL_FILE, engine="openpyxl") as writer:

    chunks_df.to_excel(
        writer,
        sheet_name="Generated_Chunks",
        index=False
    )
```

```
        worksheet = writer.sheets["Generated_Chunks"]

        # Header formatting
        for cell in worksheet[1]:
            cell.font = Font(bold=True, color="FFFFFF")
            cell.fill = PatternFill(
                fill_type="solid",
                fgColor="1F4E78"
            )
            cell.alignment = Alignment(
                horizontal="center",
                vertical="center",
                wrap_text=True
            )

        # Column widths
        column_widths = {
            "A": 12,
            "B": 40,
            "C": 14,
            "D": 18,
            "E": 20,
            "F": 23,
            "G": 80
        }

        for column, width in column_widths.items():
            worksheet.column_dimensions[column].width = width

        # Cell formatting
        for row in worksheet.iter_rows(
            min_row=2,
            max_row=worksheet.max_row
        ):
            for cell in row:
                cell.alignment = Alignment(
                    vertical="top",
                    wrap_text=(cell.column == 7)
                )

        worksheet.freeze_panes = "A2"
        worksheet.auto_filter.ref = worksheet.dimensions
        worksheet.row_dimensions[1].height = 32

    print("Excel file created:", EXCEL_FILE)
    print("Chunks exported   :", len(chunks_df))

    print("\nExcel preview:")
    print(
        chunks_df[
            ["Chunk_ID", "Page_Number", "Character_Count"]
        ].head(10).to_string(index=False)
    )

    files.download(EXCEL_FILE)
```

```
files.download(EXCEL_FILE)


# ============================================================
# 11. DISPLAY SAMPLE CHUNKS
# ============================================================

print("\n" + "=" * 70)
print("SAMPLE CHUNKS")
print("=" * 70)

for chunk in chunks[:3]:
    print(f"\nChunk ID : {chunk['chunk_id']}")
    print(f"Page     : {chunk['page']}")
    print(f"Characters: {len(chunk['text'])}")
    print("-" * 70)
    print(chunk["text"])


# ============================================================
# 12. LOAD EMBEDDING MODEL
# ============================================================

print("\n" + "=" * 70)
print("LOADING EMBEDDING MODEL")
print("=" * 70)

embedding_model = SentenceTransformer(EMBEDDING_MODEL_NAME)

print("Embedding model loaded:", EMBEDDING_MODEL_NAME)


# ============================================================
# 13. PREPARE CHUNK TEXT
# ============================================================

chunk_texts = [chunk["text"] for chunk in chunks]


# ============================================================
# 14. CREATE DOCUMENT EMBEDDINGS
# ============================================================

print("\n" + "=" * 70)
print("CREATING DOCUMENT EMBEDDINGS")
print("=" * 70)

indexing_start_time = time.time()

embeddings = embedding_model.encode(
    chunk_texts,
    convert_to_numpy=True,
    normalize_embeddings=True,
    show_progress_bar=True
).astype("float32")
```

```
).astype("float32")

print("Embedding matrix shape:", embeddings.shape)


# ============================================================
# 15. CREATE FAISS INDEX
# ============================================================

print("\n" + "=" * 70)
print("CREATING FAISS VECTOR STORE")
print("=" * 70)

embedding_dimension = embeddings.shape[1]

index = faiss.IndexFlatIP(embedding_dimension)
index.add(embeddings)

INDEXING_TIME = time.time() - indexing_start_time

print("Embedding dimension:", embedding_dimension)
print("Vectors stored     :", index.ntotal)
print(f"Indexing Time      : {INDEXING_TIME:.2f} seconds")


# ============================================================
# 16. CREATE CHUNK LOOKUP
# ============================================================

chunk_lookup = {
    position: chunk
    for position, chunk in enumerate(chunks)
}


# ============================================================
# 17. RETRIEVAL FUNCTION
# ============================================================

def retrieve_documents(query, top_k=TOP_K):

    if not query or not query.strip():
        raise ValueError("The query cannot be empty.")

    if top_k <= 0:
        raise ValueError("top_k must be greater than 0.")

    query_embedding = embedding_model.encode(
        [query],
        convert_to_numpy=True,
        normalize_embeddings=True
    ).astype("float32")

    search_k = min(top_k, index.ntotal)
```

```
    scores, indices = index.search(
        query_embedding,
        search_k
    )

    results = []

    for rank, idx in enumerate(indices[0]):

        if idx == -1:
            continue

        chunk = chunk_lookup[int(idx)]

        results.append({
            "rank": rank + 1,
            "chunk_id": chunk["chunk_id"],
            "page": chunk["page"],
            "score": float(scores[0][rank]),
            "text": chunk["text"]
        })

    return results


# ============================================================
# 18. TEST RETRIEVAL
# ============================================================

print("\n" + "=" * 70)
print("TESTING VECTOR STORE")
print("=" * 70)

test_query = input(
    "\nEnter a test question (or press Enter to skip): "
)

if test_query.strip():

    test_results = retrieve_documents(test_query, TOP_K)

    print("\n" + "=" * 70)
    print("RETRIEVED CHUNKS")
    print("=" * 70)

    for result in test_results:
        print(f"\nRESULT {result['rank']}")
        print(f"Chunk ID   : {result['chunk_id']}")
        print(f"Page       : {result['page']}")
        print(f"Similarity : {result['score']:.4f}")
        print("-" * 70)
        print(result["text"])
```

```
# ============================================================
# 19. SAVE FAISS INDEX
# ============================================================

FAISS_FILE = "rag_faiss.index"

faiss.write_index(index, FAISS_FILE)

print("\nFAISS index saved:", FAISS_FILE)


# ============================================================
# 20. SAVE CHUNKS AND METADATA
# ============================================================

CHUNKS_FILE = "rag_chunks.pkl"

with open(CHUNKS_FILE, "wb") as f:
    pickle.dump(chunks, f)

print("Chunks saved:", CHUNKS_FILE)


# ============================================================
# 21. SAVE CONFIGURATION
# ============================================================

CONFIG_FILE = "rag_config.pkl"

config = {
    "chunk_size": CHUNK_SIZE,
    "chunk_overlap": CHUNK_OVERLAP,
    "top_k": TOP_K,
    "embedding_model": EMBEDDING_MODEL_NAME,
    "source_document": pdf_file,
    "minimum_page_text_length": MIN_PAGE_TEXT_LENGTH,
    "minimum_tail_length": MIN_TAIL_LENGTH
}

with open(CONFIG_FILE, "wb") as f:
    pickle.dump(config, f)

print("Configuration saved:", CONFIG_FILE)


# ============================================================
# 22. INDEXING COMPLETE
# ============================================================

print("\n" + "=" * 70)
print("INDEXING COMPLETE")
print("=" * 70)

print("Source document :", pdf_file)
```

```
print("Source document :", pdf_file)
print("Number of chunks:", len(chunks))
print("Vectors stored  :", index.ntotal)
print("Chunk size      :", CHUNK_SIZE)
print("Chunk overlap   :", CHUNK_OVERLAP)
print("Embedding model :", EMBEDDING_MODEL_NAME)

print("\nSaved files:")
print(" ", EXCEL_FILE)
print(" ", FAISS_FILE)
print(" ", CHUNKS_FILE)
print(" ", CONFIG_FILE)

print("\nThe cleaned vector store is ready for retrieval.")
```

Show hidden output

```
# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import time
from transformers import (
    AutoTokenizer,
    AutoModelForSeq2SeqLM
)

import torch


# ============================================================
# 2. DEVICE AND CONFIGURATION
# ============================================================

device = "cuda" if torch.cuda.is_available() else "cpu"

LLM_NAME = "google/flan-t5-large"

# FLAN-T5 models have a limited input context.
MAX_INPUT_TOKENS = 768
MAX_NEW_TOKENS = 200
RESERVED_TOKEN_MARGIN = 16

# Retrieve more passages to improve the chance of finding
# information distributed across different handbook sections.
RAG_TOP_K = 3

FALLBACK_ANSWER = (
    "I cannot find the answer in the provided handbook."
)
```

```
print("=" * 70)
print("BARANGAY VAW DESK HANDBOOK RAG CHATBOT")
print("=" * 70)

print("Device:", device)
print("Language model:", LLM_NAME)
print("Maximum input tokens:", MAX_INPUT_TOKENS)
print("Maximum output tokens:", MAX_NEW_TOKENS)
print("Retrieval Top-K:", RAG_TOP_K)


# ============================================================
# 3. LOAD THE LANGUAGE MODEL
# ============================================================

print("\nLoading language model...")

tokenizer = AutoTokenizer.from_pretrained(LLM_NAME)

llm = AutoModelForSeq2SeqLM.from_pretrained(LLM_NAME)

llm = llm.to(device)
llm.eval()

print("Language model loaded successfully.")


# ============================================================
# 4. VALIDATE THE EXISTING RETRIEVAL PIPELINE
# ============================================================

required_objects = [
    "retrieve_documents",
    "index",
    "chunks",
    "embedding_model"
]

missing_objects = [
    name for name in required_objects
    if name not in globals()
]

if missing_objects:
    raise RuntimeError(
        "The indexing pipeline must run first. Missing: "
        + ", ".join(missing_objects)
    )

if index.ntotal == 0:
    raise RuntimeError("The FAISS index contains no vectors.")

print("\nExisting retrieval pipeline detected.")
print("Indexed vectors:", index.ntotal)
```

```
print("Available chunks:", len(chunks))


# ============================================================
# 5. TOKEN COUNTING
# ============================================================

def count_tokens(text):
    return len(
        tokenizer.encode(
            text,
            add_special_tokens=True,
            truncation=False
        )
    )


# ============================================================
# 6. PRIORITIZE RETRIEVED DOCUMENTS
# ============================================================

def prioritize_documents(question, retrieved_documents):
    """
    Sort retrieved passages by descending similarity score.

    FAISS uses normalized embeddings and inner product
    in the indexing pipeline.
    """

    return sorted(
        retrieved_documents,
        key=lambda result: result.get(
            "score",
            float("-inf")
        ),
        reverse=True
    )


# ============================================================
# 7. BUILD THE HANDBOOK-SPECIFIC PROMPT
# ============================================================

def create_rag_prompt(question, retrieved_documents):
    """
    Construct a token-limited prompt for the Barangay
    VAW Desk Handbook.

    Answers must be supported by the retrieved passages.
    """

    ranked_documents = prioritize_documents(
        question,
        retrieved_documents
```

```
        )

        instruction = (
            "You are an information assistant about the "
            "Barangay VAW Desk Handbook, 2023 English edition. "
            "Answer the user's question using only the supplied "
            "handbook passages. The question may concern the "
            "role and responsibilities of a Barangay VAW Desk, "
            "handling reports, intake and documentation, "
            "referrals, support services, confidentiality, "
            "coordination, or procedures described in the "
            "handbook. These are possible topics, not facts "
            "that may be assumed without evidence. "
            "Give a complete, clear answer. For multi-part "
            "questions, address each part separately. Use "
            "numbered steps or bullets when appropriate. "
            "Preserve the handbook's terminology and distinguish "
            "required procedures from recommendations when the "
            "passages make that distinction. Do not invent "
            "steps, deadlines, forms, services, responsibilities, "
            "or legal requirements. Do not use outside knowledge. "
            "If the evidence is insufficient, state exactly: "
            "'I cannot find the answer in the provided handbook.' "
            "Mention the PDF page number when available. "
            "Provide information only; do not make decisions "
            "about a specific person's case."
        )

        prompt_prefix = (
            f"Question: {question}\n\n"
            f"Instructions: {instruction}\n\n"
            "Evidence from the handbook:\n"
        )

        prompt_suffix = "\n\nAnswer:"

        prefix_tokens = tokenizer.encode(
            prompt_prefix,
            add_special_tokens=False
        )

        suffix_tokens = tokenizer.encode(
            prompt_suffix,
            add_special_tokens=False
        )

        available_context_tokens = (
            MAX_INPUT_TOKENS
            - len(prefix_tokens)
            - len(suffix_tokens)
            - RESERVED_TOKEN_MARGIN
            - 1
        )
```

```
        if available_context_tokens <= 0:
            raise ValueError(
                "The question and instructions leave insufficient "
                "space for evidence. Shorten the question."
            )

        selected_parts = []
        remaining_tokens = available_context_tokens

        for result in ranked_documents:

            if remaining_tokens <= 0:
                break

            page = result.get("page", "Unknown")
            chunk_id = result.get("chunk_id", "Unknown")
            source_text = result.get("text", "").strip()

            if not source_text:
                continue

            # Preserve source identifiers in the prompt.
            labeled_text = (
                f"[PDF Page {page}; Chunk {chunk_id}]\n"
                f"{source_text}"
            )

            separator_tokens = tokenizer.encode(
                "\n\n",
                add_special_tokens=False
            ) if selected_parts else []

            source_tokens = tokenizer.encode(
                labeled_text,
                add_special_tokens=False
            )

            tokens_available = (
                remaining_tokens - len(separator_tokens)
            )

            if tokens_available <= 0:
                break

            included_tokens = source_tokens[:tokens_available]

            if not included_tokens:
                continue

            included_text = tokenizer.decode(
                included_tokens,
                skip_special_tokens=True,
                clean_up_tokenization_spaces=False
            )
```

```
            selected_parts.append(included_text)

            remaining_tokens -= (
                len(separator_tokens) + len(included_tokens)
            )

            # Stop when a passage has been truncated.
            if len(included_tokens) < len(source_tokens):
                break

        context = "\n\n".join(selected_parts)

        prompt = (
            prompt_prefix
            + context
            + prompt_suffix
        )

        final_token_count = count_tokens(prompt)

        if final_token_count > MAX_INPUT_TOKENS:
            raise ValueError(
                f"Prompt contains {final_token_count} tokens, "
                f"exceeding the configured limit of "
                f"{MAX_INPUT_TOKENS}."
            )

        print("\n--- PROMPT DIAGNOSTICS ---")
        print("Question:", question)
        print("Retrieved passages:", len(retrieved_documents))
        print("Passages included:", len(selected_parts))
        print("Prompt token count:", final_token_count)
        print("Input token limit:", MAX_INPUT_TOKENS)

        print("\n--- EVIDENCE INCLUDED IN PROMPT ---")

        for i, part in enumerate(selected_parts, start=1):
            print(f"\nEvidence {i}:")
            print(part)

        return prompt


    # ============================================================
    # 8. GENERATE THE ANSWER
    # ============================================================

    def generate_answer(prompt, max_new_tokens=MAX_NEW_TOKENS):
        """
        Generate an answer within the configured output limit.
        """

        inputs = tokenizer(
```

```
            prompt,
            return_tensors="pt",
            truncation=False
        )

        input_length = inputs["input_ids"].shape[1]

        print("\n--- GENERATION DIAGNOSTICS ---")
        print("Input tokens:", input_length)
        print("Input limit:", MAX_INPUT_TOKENS)
        print("Maximum new tokens:", max_new_tokens)

        if input_length > MAX_INPUT_TOKENS:
            raise ValueError(
                f"Input contains {input_length} tokens, exceeding "
                f"the configured limit of {MAX_INPUT_TOKENS}."
            )

        inputs = {
            key: value.to(device)
            for key, value in inputs.items()
        }

        with torch.no_grad():
            outputs = llm.generate(
                **inputs,
                max_new_tokens=max_new_tokens,
                do_sample=False,
                num_beams=4,
                early_stopping=True
            )

        answer = tokenizer.decode(
            outputs[0],
            skip_special_tokens=True
        ).strip()

        if not answer:
            return (
                "The model generated an empty answer. "
                "Check the prompt and retrieved passages."
            )

        return answer


    # ============================================================
    # 9. COMPLETE RAG QUERY
    # ============================================================

    def rag_query(question, top_k=RAG_TOP_K):
        """
        1. Retrieve handbook passages.
        2. Build a token-limited prompt.
```

```
    3. Generate an answer.
    4. Return answer, sources, and prompt.
    """

    question = question.strip()

    if not question:
        return "Please enter a question.", [], ""

    retrieved_documents = retrieve_documents(
        question,
        top_k=top_k
    )

    if not retrieved_documents:
        return FALLBACK_ANSWER, [], ""

    prompt = create_rag_prompt(
        question,
        retrieved_documents
    )

    answer = generate_answer(prompt)

    if not answer or not answer.strip():
        answer = (
            "The model could not generate an answer. "
            "Please inspect the retrieved passages."
        )

    return answer.strip(), retrieved_documents, prompt


# ============================================================
# 10. CONTINUOUS QUESTION-ANSWERING LOOP
# ============================================================

print("\n" + "=" * 70)
print("CONTINUOUS HANDBOOK QUESTION ANSWERING")
print("=" * 70)

print("Ask questions about the Barangay VAW Desk Handbook.")
print("Type 'exit' to stop.")
print("Type 'sources' to show the last query's retrieved passages."
print("Type 'prompt' to show the last prompt.")

last_answer = None
last_sources = []
last_prompt = None

query_response_times = []

while True:

    print()
```

```
        print()
        user_question = input("Question: ").strip()

        if user_question.lower() == "exit":
            print("\nRAG session terminated.")

            print("\n" + "=" * 70)
            print("PERFORMANCE SUMMARY")
            print("=" * 70)

            if "INDEXING_TIME" in globals():
                print(f"Indexing Time            : {INDEXING_TIME:.2f}

            if query_response_times:
                avg_response_time = sum(query_response_times) / len(que
                print(f"Total Queries Answered   : {len(query_response_
                print(f"Average Response Time    : {avg_response_time:.
            else:
                print("Average Response Time    : N/A (No queries execu

            if 'script_start_time' in globals():
                total_execution_time = time.time() - script_start_time
                print(f"Total Execution Time     : {total_execution_tim

            print("=" * 70)
            break

        if not user_question:
            print("Please enter a question.")
            continue

        if user_question.lower() == "sources":

            if not last_sources:
                print("No previous query sources are available.")
                continue

            print("\n" + "=" * 70)
            print("SOURCES FROM LAST QUERY")
            print("=" * 70)

            for result in last_sources:

                score = result.get("score", None)
                score_text = (
                    f"{score:.4f}"
                    if isinstance(score, (int, float))
                    else "Unavailable"
                )

                print(
                    f"\nRank {result.get('rank', '?')} | "
                    f"PDF Page {result.get('page', '?')} | "
                    f"Chunk {result.get('chunk_id', '?')} | "
                    f"Similarity {score_text}"
```

```
                f"Similarity {score_text}"
            )

            print("-" * 70)
            print(result.get("text", ""))

        continue

    if user_question.lower() == "prompt":

        if last_prompt is None:
            print("No previous prompt is available.")
            continue

        print("\n" + "=" * 70)
        print("LAST PROMPT SENT TO THE MODEL")
        print("=" * 70)
        print(last_prompt)

        continue

    try:

        t_query_start = time.time()

        answer, retrieved_documents, prompt = rag_query(
            user_question,
            top_k=RAG_TOP_K
        )

        t_query_end = time.time()
        query_duration = t_query_end - t_query_start
        query_response_times.append(query_duration)

        last_answer = answer
        last_sources = retrieved_documents
        last_prompt = prompt

        print("\n" + "=" * 70)
        print("RAG ANSWER")
        print("=" * 70)
        print(answer)
        print(f"\n[Response Time: {query_duration:.2f}s]")

        print("\n" + "=" * 70)
        print("RETRIEVED SOURCES")
        print("=" * 70)

        if not retrieved_documents:
            print("No passages were retrieved.")

        for result in retrieved_documents:

            score = result.get("score", None)
            score_text = (
```

```
                score_text = (
                    f"{score:.4f}"
                    if isinstance(score, (int, float))
                    else "Unavailable"
                )

                print(
                    f"\n[{result.get('rank', '?')}] "
                    f"PDF Page {result.get('page', '?')} | "
                    f"Chunk {result.get('chunk_id', '?')} | "
                    f"Similarity {score_text}"
                )

                print("-" * 70)
                print(result.get("text", ""))

        except Exception as error:

            print("\nERROR:")
            print(str(error))
            print("The question could not be processed.")

    print("\n")
    print("=" * 70)
    print("RAG PROGRAM FINISHED")
    print("=" * 70)
```

Show hidden output

Tested Parameter Configurations

- The experiment has involved four different RAG setups in an effort to determine the ideal chunk size and overlaps for the two documents, namely the Barangay VAW Desk Handbook and Republic Act No. 9710 (Magna Carta of Women). The four setups were conducted using the same documents, test questions, embedding model, and generation parameters, with a Top-K of 3 being set. Retrieval performance and generated answer quality were used to evaluate the four RAG setups.

Configuration	Chunk Size	Overlap	Top-K	Purpose
A	800	80	3	Evaluate the selected chunking setup for the Bara
B	500	50	3	Evaluate smaller chunks with reduced overlap
C	1,500	300	3	Evaluate larger chunks with increased overlap for
D	300	60	3	Evaluate smaller chunks with moderate overlap fo
Selected Configuration	**800**	**80**	**3**	**Use the configuration identified as most suitab**

- The above four arrangements remained the same as those in the previous

chunking experiment. The chunk size and the overlap were calculated in terms of characters, while Top-K denoted the number of chunks obtained per query.

Compare and Analyze the Results of CPU and GPU Performance

The experiment of comparing the performance of CPU and GPU was carried out through the use of four different RAG setups. In this case, both systems utilized identical source documents, test questions, the same embedding model, and parameters for generation. The criteria that were used in measuring the performance of the two systems were indexing time, average answer time, and total execution time. Through these criteria, the difference in processing efficiency of CPU and GPU execution was determined.

From the findings, there was generally an overall reduction in the total execution time by using GPU processing, but the extent of the change depended on the settings used. In the 800/80 setting, for instance, GPU processing brought about a reduction in the total time taken from 19 minutes 26.56 seconds to 9 minutes 48.53 seconds. However, GPU had a longer average response time as compared to CPU processing, at 36.18 seconds against 32.75 seconds respectively. The same trend was seen in the 500/50 setting where CPU and GPU total execution times were more or less equal, at 10 minutes 21.06 seconds and 10 minutes 21.50 seconds respectively.

Most evident change was noticed in 1500/300 case. Time taken by GPU indexing was 5.48 seconds as compared to 1 minute 0.13 seconds taken by CPU. Average response time was lowered down from 44.00 seconds to 2.71 seconds. Likewise in the 300/60 case, time taken by GPU was reduced from 1 minute 33.83 seconds to 2.62 seconds and average response time from 23.74 seconds to 2.18 seconds.

All in all, the analysis indicates that GPU acceleration is able to increase the efficiency of the processing of the RAG in some cases, especially with the use of the 1500/300 and 300/60 combinations. On the other hand, the data indicate that even if the GPU performs faster than the CPU, it does not always mean that the GPU will perform better in all configurations and tests. The 800/80 and 500/50 configurations were characterized by slower responses in terms of GPU and almost the same execution time in 500/50 for both machines.

Experiment Results: CPU and GPU Performance

Device	Chunk Size	Overlap	Top-K	Indexing Time	Average Response Time	Total Execution Time
CPU	800	80	3	1 min 5.57 sec	32.75 sec	19 min 26.56 sec
GPU	800	80	3	1 min 1.29 sec	36.18 sec	9 min 48.53 sec
CPU	500	50	3	1 min 11.19 sec	24.34 sec	10 min 21.06 sec
GPU	500	50	3	1 min 18.60 sec	25.41 sec	10 min 21.50 sec
CPU	1,500	300	3	1 min 0.13 sec	44.00 sec	11 min 5.25 sec
GPU	1,500	300	3	5.48 sec	2.71 sec	2 min 53.79 sec
CPU	300	60	3	1 min 33.83 sec	23.74 sec	8 min 10.49 sec
GPU	300	60	3	2.62 sec	2.18 sec	2 min 46.30 sec

Performance Metrics and Speedup Calculation

- Average response time is computed as the total time taken to answer the nine test questions divided by the number of questions.

$$\text{Average Response Time} = \frac{\text{Total Time for Answering Questions}}{\text{Number of Questions}}$$

- The speedup was computed using the formula CPU execution time divided by GPU execution time. Then it is categorized as either Speedup greater than 1: GPU is faster, Speedup equal to 1: Both devices have approximately the same execution time, Speedup less than 1: CPU is faster for that measurement.

$$\boxed{\text{Speedup} = \frac{T_{\text{CPU}}}{T_{\text{GPU}}}}$$

Recommended Configuration

Based on the experimental results, the recommended configuration for the Retrieval-Augmented Generation (RAG) chatbot using the Barangay VAW Desk Handbook and Republic Act No. 9710 (Magna Carta of Women) is a chunk size of 800, an overlap of 80, a Top-K value of 3, and GPU execution using the `sentence-transformers/all-MiniLM-L6-v2` embedding model with FAISS for vector storage. This configuration was selected based on the combined evaluation of retrieval performance, generated answer quality, and processing efficiency.

During CPU and GPU benchmarking, the selected 800/80 configuration recorded an average response time of 32.75 seconds on CPU and 36.18 seconds on GPU. However, GPU execution reduced the total execution time from 19 minutes and 26.56 seconds to 9 minutes and 48.53 seconds. This represents an overall execution speedup of approximately 1.98 times. Although GPU processing had a slightly longer average response time, it completed the overall execution in significantly less time.

The chunk size of 800 and overlap of 80 were retained as the selected retrieval settings, while Top-K 3 limits retrieval to three document chunks per query. This provides the language model with a focused set of retrieved information to support answer generation. The selection was based on the combined assessment of quantitative retrieval metrics and the relevance and usefulness of the retrieved passages. However, this configuration should be considered the most suitable among the tested setups for the project's datasets and experimental conditions rather than a universally optimal configuration for all RAG applications.

Evaluation Results of the Barangay VAW Desk Handbook and Republic Act No. 9710 RAG Chatbot

Evaluation of the RAG chatbot was done through the use of twelve questions formulated based on two knowledge bases: Barangay VAW Desk Handbook (2023) and Republic Act No. 9710 or Magna Carta of Women. Questions were divided into three different difficulties: Easy (Q1-Q4), Average (Q5-Q8), and Difficult (Q9-Q12). The evaluation process involved testing the capacity of the chatbot to search for information related to its obligations as a VAW Desk staff member, the procedures in handling victims and survivors, documentation, referrals, and women's rights.

The output responses proved that the chatbot can deliver relevant and accurate information for a number of questions. For instance, Q2 delivered the right information as required in an interview including the victim-survivor's personal information, purpose of visit, and actions to take. The same case applies to Q3 which delivered the correct short title of Republic Act No. 9710 which is "Magna Carta of Women." Q9 also produced a very elaborate answer on how to create a safe and confidential environment, attend to the immediate needs, conduct risk assessment, seek medical or protective help, and fill the intake form.

Some of the answers provided, however, had errors or omissions. In Q1, while the chatbot provided help in getting BPOs for the victims as well as their access to relevant services, it did not clarify the full range of the responsibilities of the VAW Desk. In Q5, it was correctly pointed out that the victim-survivor should be in a state where he/she can undergo the interview, although it did not include the

necessary use of the referral form. In Q6 and Q10, the need for referral of victim-survivors to relevant institutions/agencies was pointed out without explaining how to identify their needs and available referral mechanisms.

There were also problems faced by the chatbot in dealing with legal provisions in answering some questions. In Q4, the chatbot erroneously answered that CEDAW is the Council for the Welfare of Children while the correct answer is the Convention on the Elimination of all forms of Discrimination against Women. On Q7, the answer included a generic statement concerning an international convention on human rights but did not give the specific convention asked by the question. On Q8, the chatbot gave the correct five years duration but omitted to include the target percentage as required. On Q11, the answer stated that the reporting bodies are the National Government Agencies (NGAs) and other government instrumentalities who submit the reports annually to the Philippine Commission on Women (PCW), however, the answer did not include the specific information as required on the second report. Lastly, on Q12, the chatbot was not able to find any answers from the handbook given.

All in all, it became clear that the chatbot coped with the questions that implied retrieval of certain facts and procedures better, especially those that concerned helping the victims. At the same time, the chatbot had difficulties in providing answers that involved certain terms of the law, numbers and the use of information from specific parts of the documents used for creating the knowledge base.

Initial Observation

- Overall, the results show that the chatbot was able to provide information and appropriate responses to some of the questions asked. However, some of the responses were missing key details, whereas others provided either wrong or partial information, especially where the legal provisions were concerned.
- These findings demonstrate that the performance of the chatbot should not be measured based on the relevance of the information but also on its accuracy, completeness, and consistency with the source documents. Missing information can be significant in terms of use of the response provided.
- As far as the computational efficiency is concerned, GPU execution shortened the total duration of execution for most of the tested configurations. Nevertheless, faster computations do not guarantee higher quality of the answers because the accuracy and thoroughness of responses also rely on the settings of information retrieval, contextual data, and

generation parameters.

- In light of the fact that only twelve questions were tested, the results should be regarded as preliminary. It is recommended to run the test on more questions in order to check the reliability of the bot.